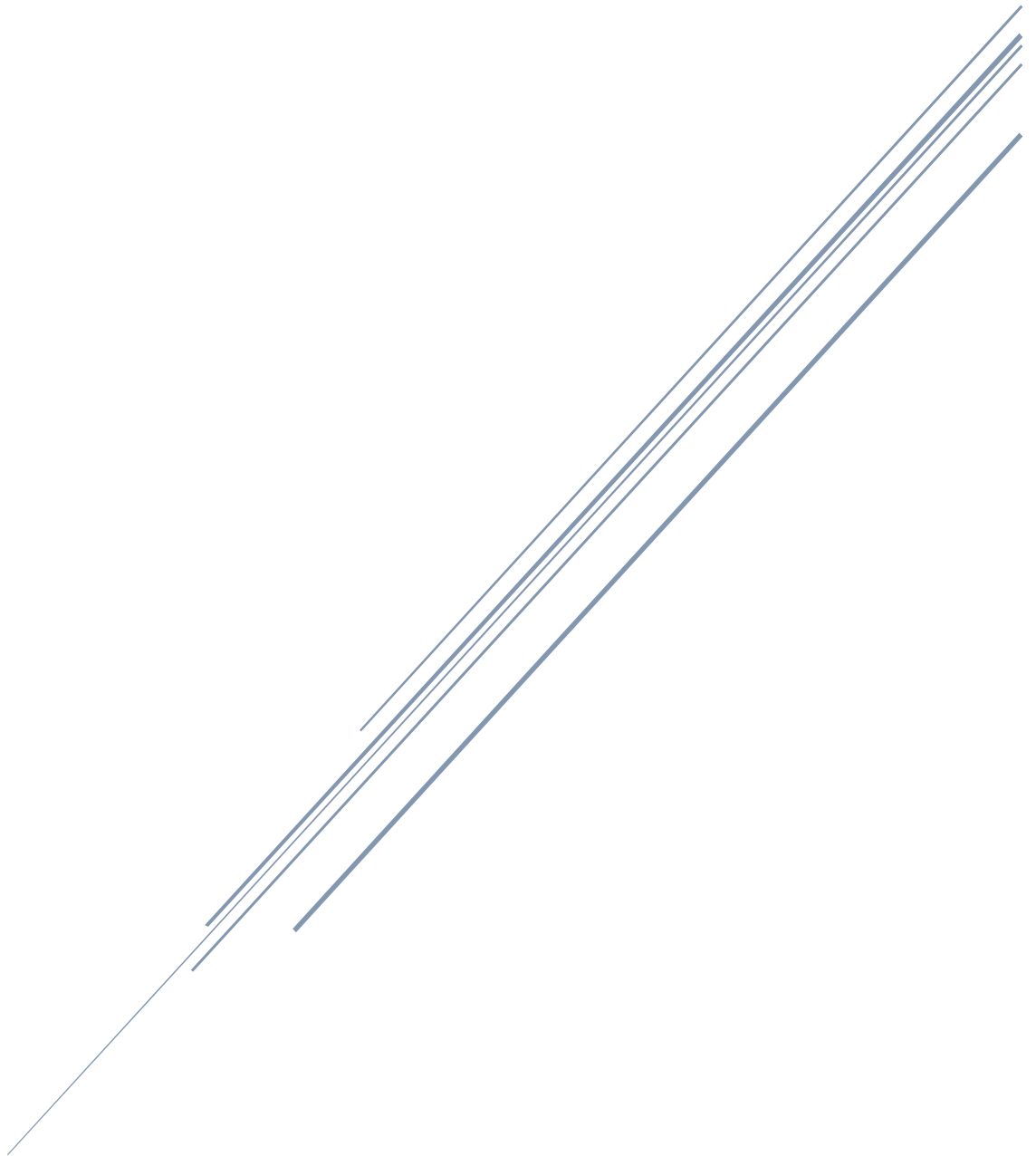


CHAPTER 7

Loss functions



The last chapters described linear regression, polynomial regression, shallow neural networks, and deep neural networks. Each represents a family of functions that map input to output, where the particular member of the family is determined by the model parameters ϕ . When we train these models, we seek the parameters that produce the best possible mapping from input to output for the task we are considering. This chapter defines what is meant by the “best possible” mapping.

That definition requires a training dataset $\{x_i, y_i\}$ of input/output pairs. A loss function or cost function $L[\phi]$ returns a single number that describes the mismatch between the model predictions $f[x_i, \phi]$ and their corresponding ground-truth outputs y_i . During training, we seek parameter values ϕ that minimize the loss and hence map the training inputs to the outputs as closely as possible. We saw one example of a loss function; the least squares loss function is suitable for univariate regression problems for which the target is a real number $y \in \mathbb{R}$. It computes the sum of the squares of the deviations between the model predictions $f[x_i, \phi]$ and the true values y_i .

This chapter provides a framework that both justifies the choice of the least squares criterion for real-valued outputs and allows us to build loss functions for other prediction types. We consider binary classification, where the prediction $y \in \{0, 1\}$ is one of two categories, multiclass classification, where the prediction $y \in \{1, 2, \dots, K\}$ is one of K categories, and more complex cases.

7.1 Maximum likelihood

In this section, we develop a recipe for constructing loss functions. Consider a model $f[x, \phi]$ with parameters ϕ that computes an output from input x . Until now, we have implied that the model directly computes a prediction y . We now shift perspective and consider the model as computing a conditional probability distribution $\Pr(y|x)$ over possible outputs y given input x . The loss encourages each training output y_i to have a high probability under the distribution $\Pr(y_i|x_i)$ computed from the corresponding input x_i (figure 7.1).

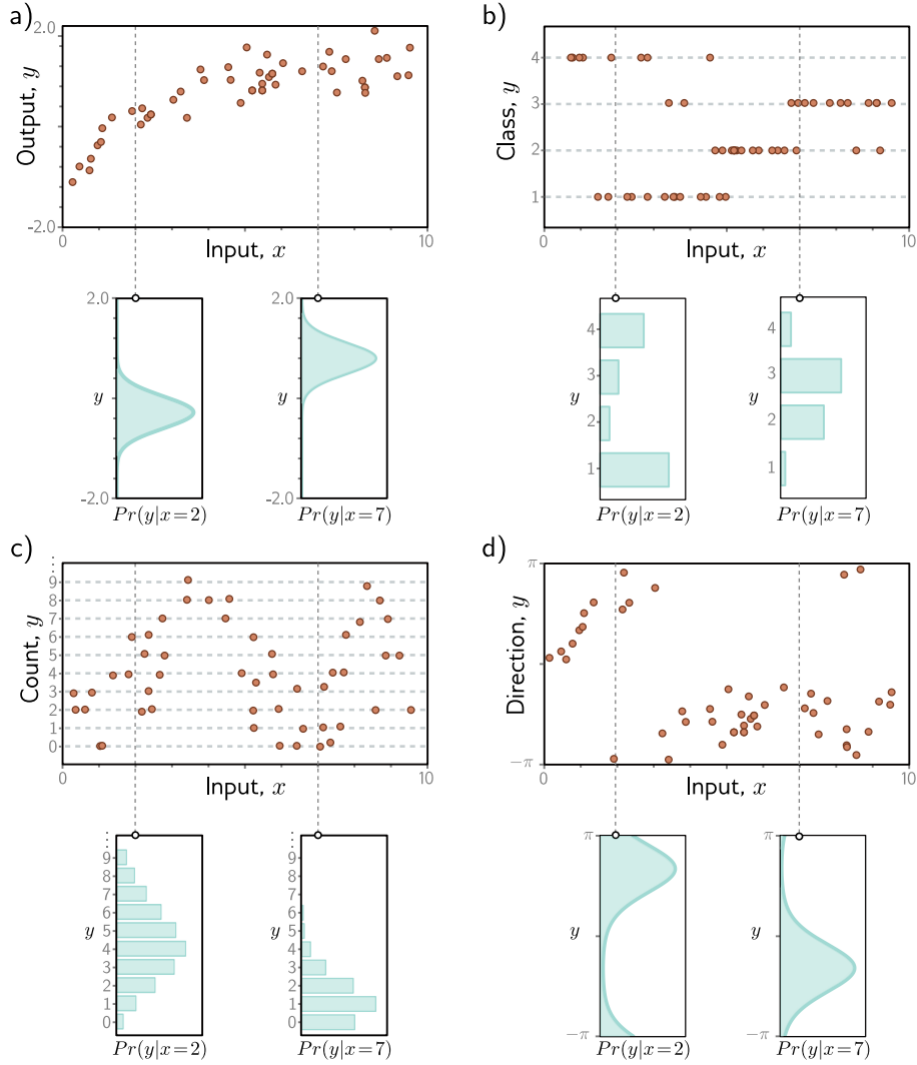


Figure 7.1 Predicting distributions over outputs. a) Regression task, where the goal is to predict a real-valued output y from the input x based on training data $\{x, y\}$ (orange points). For each input value x , the machine learning model predicts a distribution $\Pr(y|x)$ over the output $y \in \mathbb{R}$ (cyan curves show distributions for $x = 2.0$ and $x = 7.0$). The loss function aims to maximize the probability of the observed training outputs y_i under the distribution predicted from the corresponding inputs x_i . b) To predict discrete classes $y \in \{1, 2, 3, 4\}$ in a classification task, we use a discrete probability distribution, so the model predicts a different histogram over the four possible values of y_i for each value of x_i . c) To predict counts $y \in \{0, 1, 2, \dots\}$ and d) direction $y \in (-\pi, \pi]$, we use distributions defined over positive integers and circular domains, respectively.

7.2 Computing a distribution over outputs

This shift in perspective raises the question of exactly how a model $f[x, \phi]$ can be adapted to compute a probability distribution. The solution is simple. First, we choose a parametric distribution $\Pr(y|\theta)$ defined on the output domain y . Then we use the network to compute one or more of the parameters θ of this distribution.

For example, suppose the prediction domain is the set of real numbers, so $y \in \mathbb{R}$. Here, we might choose the univariate normal distribution, which is defined on $\mathbb{R}$. This distribution is defined by the mean μ and variance σ_2 , so $\theta = \{\mu, \sigma_2\}$. The machine learning model might predict the mean μ , and the variance σ_2 could be treated as an unknown constant.

7.3 Maximum likelihood criterion

The model now computes different distribution parameters $\theta_i = f[\mathbf{x}_i, \phi]$ for each training input $\mathbf{x}_i$. Each observed training output y_i should have high probability under its corresponding distribution $\Pr(y_i|\theta_i)$. Hence, we choose the model parameters ϕ so that they maximize the combined probability across all I training examples:

$$\begin{aligned}\hat{\phi} &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \Pr(y_i|\mathbf{x}_i) \right] \\ &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \Pr(y_i|\theta_i) \right] \\ &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \Pr(y_i|f[\mathbf{x}_i, \phi]) \right].\end{aligned}$$

The combined probability term is the likelihood of the parameters, and hence the above equation is known as the maximum likelihood criterion.

Here we are implicitly making two assumptions. First, we assume that the data are identically distributed (the form of the probability distribution over the outputs y_i is the same for each data point). Second, we assume that the conditional distributions $\Pr(y_i|\mathbf{x}_i)$ of the output given the input are independent, so the total likelihood of the training data decomposes as:

$$\Pr(y_1, y_2, \dots, y_I | \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_I) = \prod_{i=1}^I \Pr(y_i | \mathbf{x}_i).$$

In other words, we assume the data are independent and identically distributed (i.i.d.).

7.4 Maximizing log-likelihood

The maximum likelihood criterion is not very practical. Each term $\Pr(y_i|f[\mathbf{x}_i, \phi])$ can be small, so the product of many of these terms can be tiny. It may be difficult to represent this quantity with finite precision arithmetic. Fortunately, we can equivalently maximize the logarithm of the likelihood:

$$\begin{aligned}\hat{\phi} &= \operatorname{argmax}_{\phi} \left[\prod_{i=1}^I \Pr(y_i|f[\mathbf{x}_i, \phi]) \right] \\ &= \operatorname{argmax}_{\phi} \left[\log \left[\prod_{i=1}^I \Pr(y_i|f[\mathbf{x}_i, \phi]) \right] \right] \\ &= \operatorname{argmax}_{\phi} \left[\sum_{i=1}^I \log \left[\Pr(y_i|f[\mathbf{x}_i, \phi]) \right] \right].\end{aligned}$$

This log-likelihood criterion is equivalent because the logarithm is a monotonically increasing function: if $z > z'$, then $\log[z] > \log[z']$ and vice versa (figure 7.2). It follows that when we change the model parameters ϕ to improve the log-likelihood criterion, we also improve the original maximum likelihood criterion. It also follows that the overall maxima of the two criteria must be in the same place, so the best model parameters $\hat{\phi}$ are the same in both cases. However, the log-likelihood criterion has the practical advantage of using a sum of terms, not a product, so representing it with finite precision isn't problematic.

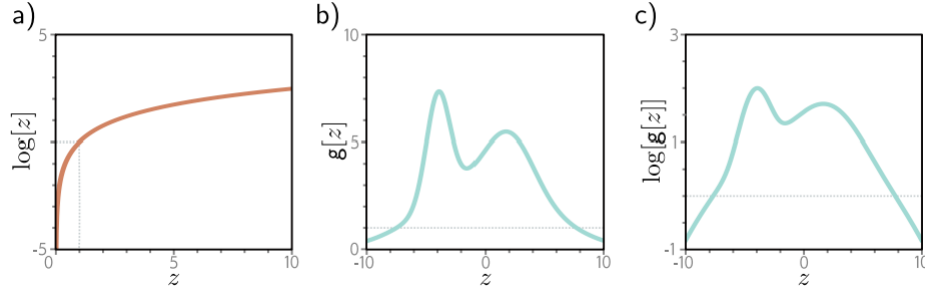


Figure 7.2 The log transform. a) The log function is monotonically increasing. If $z > z'$, then $\log[z] > \log[z']$. It follows that the maximum of any function $g[z]$ will be at the same position as the maximum of $\log[g[z]]$. b) A function $g[z]$. c) The logarithm of this function $\log[g[z]]$. All positions on $g[z]$ with a positive slope retain a positive slope after the log transform, and those with a negative slope retain a negative slope. The position of the maximum remains the same.

7.5 Minimizing negative log-likelihood

Finally, we note that, by convention, model fitting problems are framed in terms of minimizing a loss. To convert the maximum log-likelihood criterion to a minimization problem, we multiply by minus one, which gives us the negative log-likelihood criterion:

$$\begin{aligned}\hat{\phi} &= \underset{\phi}{\operatorname{argmin}} \left[-\sum_{i=1}^I \log \left[\Pr(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right] \\ &= \underset{\phi}{\operatorname{argmin}} [L[\phi]],\end{aligned}$$

which is what forms the final loss function $L[\phi]$.

7.6 Inference

The network no longer directly predicts the outputs y but instead determines a probability distribution over y . When we perform inference, we often want a point estimate rather than a distribution, so we return the maximum of the distribution:

$$\hat{\mathbf{y}} = \underset{\mathbf{y}}{\operatorname{argmax}} \left[\Pr(\mathbf{y} | \mathbf{f}[\mathbf{x}, \hat{\phi}]) \right].$$

It is usually possible to find an expression for this in terms of the distribution parameters θ predicted by the model. For example, in the univariate normal distribution, the maximum occurs at the mean μ .

7.7 Recipe for constructing loss functions

The recipe for constructing loss functions for training data $\{\mathbf{x}_i, \mathbf{y}_i\}$ using the maximum likelihood approach is hence:

1. Choose a suitable probability distribution $\Pr(\mathbf{y}|\theta)$ defined over the domain of the predictions $\mathbf{y}$ with distribution parameters θ .
2. Set the machine learning model $\mathbf{f}[\mathbf{x}, \phi]$ to predict one or more of these parameters, so $\theta = \mathbf{f}[\mathbf{x}, \phi]$ and $\Pr(\mathbf{y}|\theta) = \Pr(\mathbf{y}|\mathbf{f}[\mathbf{x}, \phi])$.
3. To train the model, find the network parameters $\hat{\phi}$ that minimize the negative log-likelihood loss function over the training dataset pairs $\{\mathbf{x}_i, \mathbf{y}_i\}$:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} [L[\phi]] = \underset{\phi}{\operatorname{argmin}} \left[-\sum_{i=1}^I \log \left[\Pr(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] \right].$$

1. To perform inference for a new test example x , return either the full distribution $\Pr(y|f[x, \hat{\phi}])$ or the value where this distribution is maximized.

7.8 Examples

Example 1: univariate regression

We start by considering univariate regression models. Here the goal is to predict a single scalar output $y \in \mathbb{R}$ from input x using a model $f[x, \phi]$ with parameters ϕ . Following the recipe, we choose a probability distribution over the output domain y . We select the univariate normal (figure 7.3), which is defined over $y \in \mathbb{R}$. This distribution has two parameters (mean μ and variance σ^2) and has a probability density function:

$$\Pr(y|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y - \mu)^2}{2\sigma^2} \right].$$

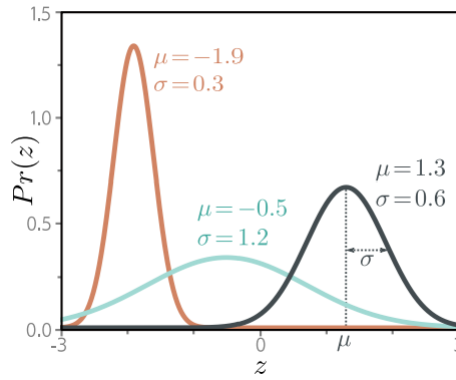


Figure 7.3 The univariate normal distribution (also known as the Gaussian distribution) is defined on the real line $z \in \mathbb{R}$ and has parameters μ and σ^2 . The mean μ determines the position of the peak. The positive root of the variance σ^2 (the standard deviation) determines the width of the distribution. Since the total probability density sums to one, the peak becomes higher as the variance decreases and the distribution becomes narrower.

Second, we set the machine learning model $f[x, \phi]$ to compute one or more of the parameters of this distribution. Here, we just compute the mean so $\mu = f[x, \phi]$:

$$\Pr(y|f[x, \phi], \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y - f[x, \phi])^2}{2\sigma^2} \right].$$

We aim to find the parameters ϕ that make the training data $\{x_i, y_i\}$ most probable under this distribution (figure 7.4). To accomplish this, we choose a loss function $L[\phi]$ based on the negative log-likelihood:

$$\begin{aligned} L[\phi] &= -\sum_{i=1}^I \log [\Pr(y_i | f[x_i, \phi], \sigma^2)] \\ &= -\sum_{i=1}^I \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - f[x_i, \phi])^2}{2\sigma^2} \right] \right]. \end{aligned}$$

When we train the model, we seek parameters $\hat{\phi}$ that minimize this loss.

Least squares loss function:

Now let's perform some algebraic manipulations on the loss function. We seek:

$$\begin{aligned}
\hat{\phi} &= \underset{\phi}{\operatorname{argmin}} \left[-\sum_{i=1}^I \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - f[\mathbf{x}_i, \phi])^2}{2\sigma^2} \right] \right] \right] \\
&= \underset{\phi}{\operatorname{argmin}} \left[-\sum_{i=1}^I \left(\log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \right] - \frac{(y_i - f[\mathbf{x}_i, \phi])^2}{2\sigma^2} \right) \right] \\
&= \underset{\phi}{\operatorname{argmin}} \left[-\sum_{i=1}^I -\frac{(y_i - f[\mathbf{x}_i, \phi])^2}{2\sigma^2} \right] \\
&= \underset{\phi}{\operatorname{argmin}} \left[\sum_{i=1}^I (y_i - f[\mathbf{x}_i, \phi])^2 \right],
\end{aligned}$$

where we removed the first term between the second and third lines because it doesn't depend on ϕ . We removed the denominator between the third and fourth lines, as this is just a constant positive scaling factor that does not affect the position of the minimum. The result of these manipulations is the least squares loss function that we originally introduced when we discussed linear regression.

$$L[\phi] = \sum_{i=1}^I (y_i - f[\mathbf{x}_i, \phi])^2.$$

We see that the least squares loss function follows naturally from the assumptions that the predictions are (i) independent and (ii) drawn from a normal distribution with mean $\mu = f[\mathbf{x}_i, \phi]$ (figure 7.4).

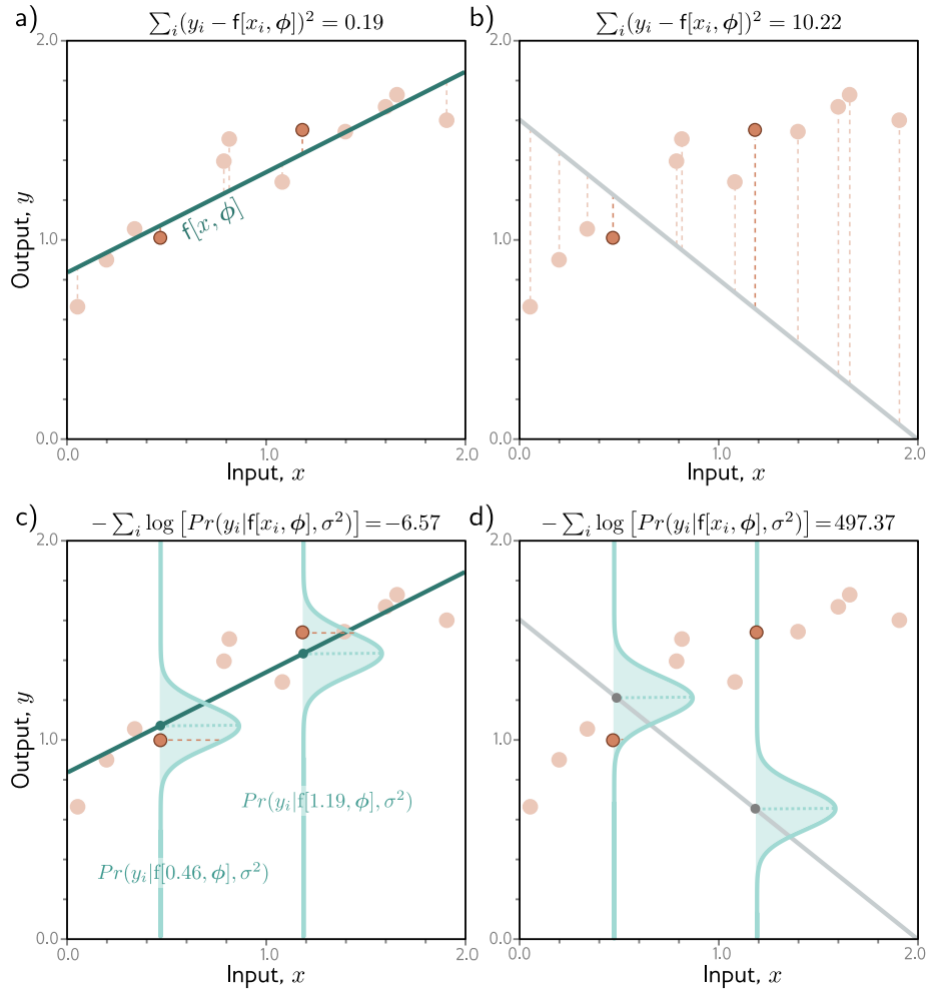


Figure 7.4 Equivalence of least squares and maximum likelihood loss for the normal distribution. a) The least squares

criterion minimizes the sum of the squares of the deviations (dashed lines) between the model prediction $f[x, \phi]$ (green line) and the true output values y_i (orange points). Here the fit is good, so these deviations are small (e.g., for the two highlighted points). b) For these parameters, the fit is bad, and the squared deviations are large. c) The least squares criterion follows from the assumption that the model predicts the mean of a normal distribution over the outputs and that we maximize the probability. For the first case, the model fits well, so the probability $Pr(y_i|x_i)$ of the data (horizontal orange dashed lines) is large (and the negative log probability is small). d) For the second case, the model fits badly, so the probability is small and the negative log probability is large.

Inference

The network no longer directly predicts y but instead predicts the mean $\mu = f[x, \phi]$ of the normal distribution over y . When we perform inference, we usually want a single “best” point estimate $\hat{y}$, so we take the maximum of the predicted distribution:

$$\hat{y} = \underset{y}{\operatorname{argmax}} \left[Pr(y|f[x, \hat{\phi}], \sigma^2) \right].$$

For the univariate normal, the maximum position is determined by the mean parameter μ (figure 7.3). This is precisely what the model computed, so $\hat{y} = f[x, \hat{\phi}]$.

Example 2: binary classification

In binary classification, the goal is to assign the data x to one of two discrete classes $y \in \{0, 1\}$. In this context, we refer to y as a label. Examples of binary classification include (i) predicting whether a restaurant review is positive ($y = 1$) or negative ($y = 0$) from text data x and (ii) predicting whether a tumor is present ($y = 1$) or absent ($y = 0$) from an MRI scan x .

Once again, we follow the recipe from section 5.2 to construct the loss function. First, we choose a probability distribution over the output space $y \in \{0, 1\}$. A suitable choice is the Bernoulli distribution, which is defined on the domain $\{0, 1\}$. This has a single parameter $\lambda \in [0, 1]$ that represents the probability that y takes the value one (figure 7.5):

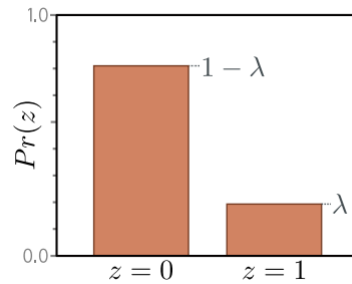


Figure 7.5 Bernoulli distribution. The Bernoulli distribution is defined on the domain $z \in \{0, 1\}$ and has a single parameter λ that denotes the probability of observing $z = 1$. It follows that the probability of observing $z = 0$ is $1 - \lambda$.

$$Pr(y|\lambda) = \begin{cases} 1 - \lambda & y = 0 \\ \lambda & y = 1 \end{cases},$$

which can equivalently be written as:

$$Pr(y|\lambda) = (1 - \lambda)^{1-y} \cdot \lambda^y.$$

Second, we set the machine learning model $f[x, \phi]$ to predict the single distribution parameter λ .

However, λ can only take values in the range $[0, 1]$, and we cannot guarantee that the network output will lie in this range. Consequently, we pass the network output through a function that maps the real numbers $\mathbb{R}$ to $[0, 1]$. A suitable function is the logistic sigmoid (figure 7.6):

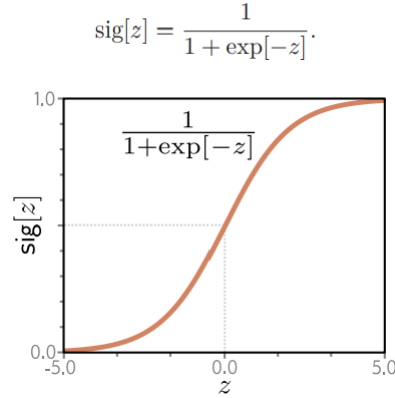


Figure 7.6 Logistic sigmoid function. This function maps the real line $z \in \mathbb{R}$ to numbers between zero and one, so $\text{sig}[z] \in [0, 1]$. An input of 0 is mapped to 0.5. Negative inputs are mapped to numbers below 0.5, and positive inputs to numbers above 0.5.

Hence, we predict the distribution parameter as $\lambda = \text{sig}[f[\mathbf{x}, \phi]]$. The likelihood is now:

$$Pr(y|\mathbf{x}) = (1 - \text{sig}[f[\mathbf{x}, \phi]])^{1-y} \cdot \text{sig}[f[\mathbf{x}, \phi]]^y.$$

This is depicted in figure 7.7 for a shallow neural network model. The loss function is the negative log-likelihood of the training set:

$$L[\phi] = \sum_{i=1}^I -(1 - y_i) \log [1 - \text{sig}[f[\mathbf{x}_i, \phi]]] - y_i \log [\text{sig}[f[\mathbf{x}_i, \phi]]].$$

For reasons to be explained later, this is known as the *binary cross-entropy loss*.

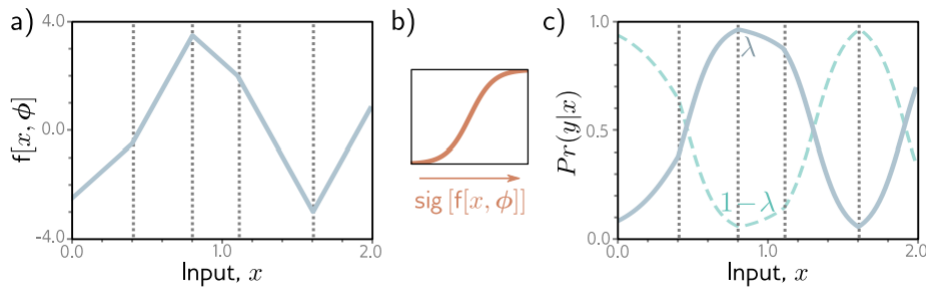


Figure 7.7 Binary classification model. a) The network output is a piecewise linear function that can take arbitrary real values. b) This is transformed by the logistic sigmoid function, which compresses these values to the range $[0, 1]$. c) The transformed output predicts the probability λ that $y = 1$ (solid line). The probability that $y = 0$ is hence $1 - \lambda$ (dashed line). For any fixed x (vertical slice), we retrieve the two values of a Bernoulli distribution. The loss function favors model parameters that produce large values of λ at positions x_i that are associated with positive examples $y_i = 1$ and small values of λ at positions associated with negative examples $y_i = 0$.

The transformed model output $\text{sig}[f[\mathbf{x}, \phi]]$ predicts the parameter λ of the Bernoulli distribution. This represents the probability that $y = 1$, and it follows that $1 - \lambda$ represents the probability that $y = 0$. When

we perform inference, we may want a point estimate of y , so we set $y = 1$ if $\lambda > 0.5$ and $y = 0$ otherwise.

Example 3: multiclass classification

The goal of multiclass classification is to assign an input data example x to one of $K > 2$ classes, so $y \in \{1, 2, \dots, K\}$. Real-world examples include (i) predicting which of $K = 10$ digits y is present in an image x of a handwritten number and (ii) predicting which of K possible words y follows an incomplete sentence x . We once more follow the recipe from section 5.2. We first choose a distribution over the prediction space y . In this case, we have $y \in \{1, 2, \dots, K\}$, so we choose the categorical distribution (figure 7.8), which is defined on this domain. This has K parameters $\lambda_1, \lambda_2, \dots, \lambda_K$, which determine the probability of each category:

$$Pr(y = k) = \lambda_k.$$

The parameters are constrained to take values between zero and one, and they must collectively sum to one to ensure a valid probability distribution.

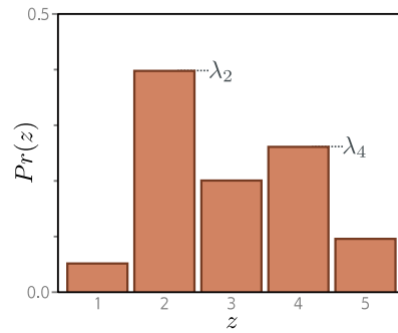


Figure 7.8 Categorical distribution. The categorical distribution assigns probabilities to $K > 2$ categories, with associated probabilities $\lambda_1, \lambda_2, \dots, \lambda_K$. Here, there are five categories, so $K = 5$. To ensure that this is a valid probability distribution, each parameter λ_k must lie in the range $[0, 1]$, and all K parameters must sum to one.

Then we use a network $f[x, \phi]$ with K outputs to compute these K parameters from the input x . Unfortunately, the network outputs will not necessarily obey the aforementioned constraints. Consequently, we pass the K outputs of the network through a function that ensures these constraints are respected. A suitable choice is the softmax function (figure 7.9). This takes an arbitrary vector of length K and returns a vector of the same length but where the elements are now in the range $[0, 1]$ and sum to one. The k^{th} output of the softmax function is:

$$\text{softmax}_k[\mathbf{z}] = \frac{\exp[z_k]}{\sum_{k'=1}^K \exp[z_{k'}]},$$

where the exponential functions ensure positivity, and the sum in the denominator ensures that the K numbers sum to one.

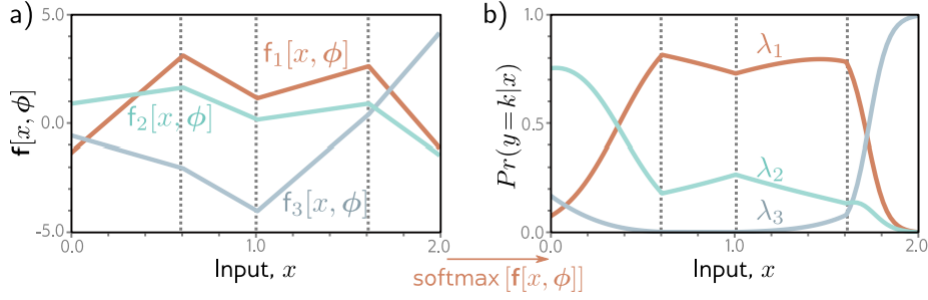


Figure 7.9 Multiclass classification for $K = 3$ classes. a) The network has three piecewise linear outputs, which can take arbitrary values. b) After the softmax function, these outputs are constrained to be non-negative and sum to one. Hence, for a given input x , we compute valid parameters for the categorical distribution: any vertical slice of this plot produces three values that sum to one and would form the heights of the bars in a categorical distribution.

The likelihood that input x has label $y = k$ (figure 7.9) is hence:

$$Pr(y = k|x) = \text{softmax}_k[\mathbf{f}[x, \phi]].$$

The loss function is the negative log-likelihood of the training data:

$$\begin{aligned} L[\phi] &= -\sum_{i=1}^I \log [\text{softmax}_{y_i}[\mathbf{f}[\mathbf{x}_i, \phi]]] \\ &= -\sum_{i=1}^I \left(f_{y_i}[\mathbf{x}_i, \phi] - \log \left[\sum_{k'=1}^K \exp[f_{k'}[\mathbf{x}_i, \phi]] \right] \right), \end{aligned}$$

where $f_{y_i}[\mathbf{x}, \phi]$ and $f_{k'}[\mathbf{x}, \phi]$ denote the y_i^{th} and k'^{th} outputs of the network, respectively. For reasons that will be explained later, this is known as the *multiclass cross-entropy loss*.

The transformed model output represents a categorical distribution over possible classes $y \in \{1, 2, \dots, K\}$. For a point estimate, we take the most probable category $\hat{y} = \text{argmax}_k Pr(y = k | \mathbf{f}[\mathbf{x}, \hat{\phi}])$. This corresponds to whichever curve is highest for that value of x in figure 7.9.

Multiple outputs

Often, we wish to make more than one prediction with the same model, so the target output y is a vector. For example, we might want to predict a molecule's melting and boiling point (a multivariate regression problem) or the object class at every point in an image (a multivariate classification problem). While it is possible to define multivariate probability distributions and use a neural network to model their parameters as a function of the input, it is more usual to treat each prediction as independent.

Independence implies that we treat the probability $Pr(y|\mathbf{f}[\mathbf{x}, \phi])$ as a product of univariate terms for each element $y_d \in y$:

$$Pr(y|\mathbf{f}[\mathbf{x}, \phi]) = \prod_d Pr(y_d | \mathbf{f}_d[\mathbf{x}, \phi]),$$

where $\mathbf{f}_d[\mathbf{x}, \phi]$ is the d^{th} set of network outputs, which describe the parameters of the distribution over y_d . For example, to predict multiple continuous variables $y_d \in \mathbb{R}$, we use a normal distribution for each y_d , and the network outputs $\mathbf{f}_d[\mathbf{x}, \phi]$ predict the means of these distributions. To predict multiple discrete variables $y_d \in \{1, 2, \dots, K\}$, we use a categorical distribution for each y_d . Here, each set of network outputs $\mathbf{f}_d[\mathbf{x}, \phi]$ predicts the K values that contribute to the categorical distribution for y_d .

When we minimize the negative log probability, this product becomes a sum of terms:

$$L[\phi] = - \sum_{i=1}^I \log \left[Pr(\mathbf{y}_i | \mathbf{f}[\mathbf{x}_i, \phi]) \right] = - \sum_{i=1}^I \sum_d \log \left[Pr(y_{id} | \mathbf{f}_d[\mathbf{x}_i, \phi]) \right].$$

where y_{id} is the d^{th} output from the i^{th} training example. To make two or more prediction types simultaneously, we similarly assume the errors in each are independent. For example, to predict wind direction and strength, we might choose the von Mises distribution (defined on circular domains) for the direction and the exponential distribution (defined on positive real numbers) for the strength. The independence assumption implies that the joint likelihood of the two predictions is the product of individual likelihoods. These terms will become additive when we compute the negative log-likelihood.